

MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Balan Gabriel FAF-241

Report

Laboratory Work No. 10

Operating Systems

Topic: Add Your Own System Call: trace

Checked by:
Reiţman Patricia, *university assistant*

Chişinău – 2026

Contents

1 Purpose of the Work 3

2 Part 1 – The Number and the User Side 3

3 Part 2 – The Kernel Side 3

4 Part 3 – Play with the Mask 5

4.1 All System Calls 5

4.2 Only fork 5

4.3 Only write 6

4.4 Observations 6

5 Conclusion 7

1. Purpose of the Work

The purpose of this laboratory work is to add a new system call, `trace`, directly into xv6. Unlike the previous laboratory, where I only observed system calls from Linux using `strace`, here I modified the xv6 user side and kernel side so a process can choose which system calls should be printed. I then tested different bit masks to trace only specific calls such as `read`, `fork`, `write`, `open` and `close`.

2. Part 1 – The Number and the User Side

The first step was to make the new system call visible from user space. The laboratory uses system call number 23 for `trace`. I added the number in `kernel/syscall.h`, declared the function in `user/user.h`, added the generated system-call stub in `user/usys.pl`, created `user/trace.c`, and added the program to `UPROGS` in the Makefile.

The most important user-side additions were:

kernel/syscall.h

```
#define SYS_trace 23
```

user/user.h and user/usys.pl

```
int trace(int);
entry("trace");
```

The `trace` user program first calls `trace(mask)` on itself and then replaces itself with the requested program using `exec`. This means the same process continues running with the tracing mask already set.

Core of user/trace.c

```
if (trace(atoi(argv[1])) < 0) {
    fprintf(2, "trace: failed\n");
    exit(1);
}
exec(argv[2], argv + 2);
```

At this stage, before the kernel handler is added, calling system call number 23 reaches the kernel but is still unknown. This is useful as an intermediate check because it proves the user-side path reaches the system-call dispatcher.

3. Part 2 – The Kernel Side

For the kernel side, every process needs to remember its own tracing mask. I added a new field to `struct proc`:

kernel/proc.h

```
int trace_mask;
```

The handler for the new system call reads the mask argument and stores it in the current process:

kernel/sysproc.c

```
uint64
sys_trace(void)
{
  int mask;
  argint(0, &mask);
  myproc()->trace_mask = mask;
  return 0;
}
```

I then connected `SYS_trace` to `sys_trace` in the syscall table and added the system-call name table. After each system call finishes, the kernel checks whether the corresponding bit is enabled in the current process mask:

kernel/syscall.c

```
p->trapframe->a0 = syscalls[num]();
if ((p->trace_mask >> num) & 1)
  printk("%d: syscall %s -> %ld\n",
         p->pid, syscallnames[num], p->trapframe->a0);
```

Finally, I made children inherit the mask during `fork`:

kernel/proc.c

```
np->trace_mask = p->trace_mask;
```

After rebuilding xv6, the required test worked correctly:

Test

```
trace 32 grep hello README
```

Output

```
3: syscall read -> 1023
3: syscall read -> 965
3: syscall read -> 453
3: syscall read -> 0
```

The first three reads add up to 2441 bytes, which is the size of `README`, and the final `0` means

end of file.

```
findredhundred@Ubuntu:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio
-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus
=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ trace 32 grep hello README
3: syscall read -> 1023
3: syscall read -> 965
3: syscall read -> 453
3: syscall read -> 0
$
```

Figure 2.1 – The completed `trace 32 grep hello README` test in xv6.

4. Part 3 – Play with the Mask

The tracing mask is treated as a bit mask. A set bit means that the system call with that number should be printed. I tested several different masks to see that only the selected calls appear.

4.1. All System Calls

Using `2147483647` enables all of the relevant bits. The output therefore includes `trace`, `exec`, `open`, the sequence of `read` calls, and `close`.

Command

```
trace 2147483647 grep hello README
```

```
$ trace 2147483647 grep hello README
3: syscall trace -> 0
3: syscall exec -> 3
3: syscall open -> 3
3: syscall read -> 1023
3: syscall read -> 965
3: syscall read -> 453
3: syscall read -> 0
3: syscall close -> 0
$
```

Figure 3.1 – Tracing all system calls used by `grep`.

4.2. Only fork

The mask `2` traces only `fork`. Running the `forkforkfork` test produces many lines from multiple processes as they continue creating children.

Command

```
trace 2 usertests forkforkfork
```

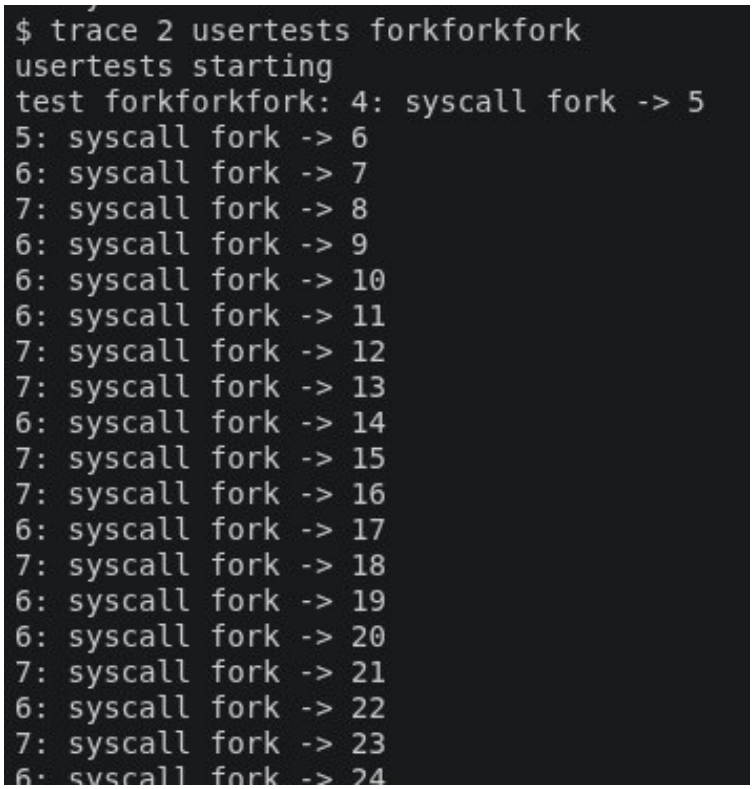


Figure 3.2 – Tracing only the `fork` system call.

4.3. Only write

The mask `65536` selects `write`. With `echo hi`, two writes are visible: one writes the characters and the other writes the newline.

Command

```
trace 65536 echo hi
```

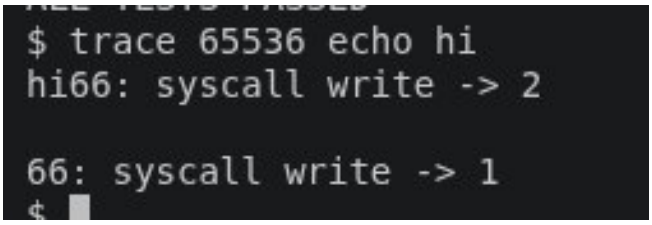


Figure 3.3 – Tracing only the `write` system call.

4.4. Observations

1. Because `32` only traces the `read` syscall, the sixth bit is on only (counting the least-significant bit as the first bit). This is why `trace 32 grep hello README` prints the

`read` calls but not `open` or `close`.

2. The line `np->trace_mask = p->trace_mask;` is what makes the output from several PIDs possible. When `fork` creates a new process, this line makes sure the mask is copied from the parent process to the child, so it is aware that it traces syscalls.

3. `echo` calls `write` twice, once to write to the screen our request, and the other to write the newline. This is why the tracing line appears between `hi` and the final newline.

4. In `kernel/syscall.h`, `SYS_open` is 15 and `SYS_close` is 21. Hence the mask needed to trace only both calls is

$$2^{15} + 2^{21} = 32768 + 2097152 = 2129920.$$

5. Conclusion

This laboratory was the first one where I added a system call directly into the xv6 kernel instead of only using the system calls that were already there. I followed the call from the user-side declaration and stub to the kernel handler, the syscall table and finally the printing logic. The mask experiment also made the bit-mask idea very clear: each syscall number corresponds to one bit, so I can choose exactly which calls should be printed. The inheritance line in `fork` was especially important, because without copying the mask the child processes would stop being traced. Overall, the lab connected user programs, system-call numbers, process state and kernel dispatching into one complete path.